



CESDE

Santiago Yosa

Ingeniero Sistemas de Información



Unidad II

JPA+HIBERNATE CONEXIÓN CON BD

- Configuración conexión con BD
- Mapeo de entidades
- Relaciones entre entidades
- Operaciones de persistencia CRUD

SISTEMA CONTROL DE VERSIONES

- Utilizar git/GitHub para versionar código realizado

Fundamentos de POO

Puente Cognitivo

Del bloque de POO, deben dominar la **Herencia (extends)** y el **Polimorfismo**. De la sesión anterior de persistencia, deben recordar qué es el **application.properties** y cómo iniciar la aplicación con el Maven Wrapper (mvnw).

```
spring.application.name=inventario
# Configuración de la base de datos SQL Server
spring.datasource.url=jdbc:sqlserver://SACHI:1433;databaseName=miinventario;trustServerCertificate=true;
spring.datasource.username=tuusuario
spring.datasource.password=tuclave
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
# Propiedades de Hibernate para JPA
# 'update' crea o actualiza las tablas automáticamente al iniciar la app
spring.jpa.hibernate.ddl-auto=update
# Muestra las sentencias SQL en la consola
spring.jpa.show-sql=true
# Especifica el dialecto de Hibernate para SQL Server
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.SQLServerDialect
```

spring.jpa.hibernate.ddl-auto:

Es una de las propiedades más importantes. Controla cómo Hibernate interactúa con el esquema de la base de datos.

- **none:** No hace nada (la opción más segura en producción).
- **update:** Actualiza el esquema de la base de datos para que coincida con las entidades de tu código.
- **create:** Borra todas las tablas y las crea de nuevo.
- **create-drop:** Hace lo mismo que create pero borra las tablas al cerrar la aplicación.

Situación

Imagina que estás desarrollando el sistema para una tienda tecnológica. Tienes una clase Padre llamada Producto (con id, precio, marca). Tienes dos clases Hijas: Computador (añade ram) y Monitor (añade pulgadas).

En el mundo de Java (POO), esto es hermoso y fácil de manejar gracias a la herencia. Pero en el mundo de las Bases de Datos Relacionales (SQL), **el concepto de Herencia no existe**. Una tabla no puede hacer extends de otra tabla.

Las tablas son planas. *¿El problema?* ¿Cómo guardamos un Computador y un Monitor en la base de datos sin perder el diseño orientado a objetos que tanto nos costó armar?

La Solución: JPA nos ofrece **Estrategias de Herencia**. Nos permite decirle a Hibernate exactamente cómo queremos que "aplaste" nuestra jerarquía de clases 3D en tablas planas 2D

Para que Hibernate convierta una clase en una tabla, necesitamos un mínimo de dos anotaciones obligatorias que importamos del paquete `jakarta.persistence.*`.

Mapeo de Entidades

@Entity: Anotación a nivel de clase que le dice a JPA: "Esta clase corresponde a una tabla en la base de datos".

@Id: Anotación a nivel de campo que marca la propiedad que será la clave primaria de la tabla. Toda entidad debe tener una.

@GeneratedValue(strategy = GenerationType.IDENTITY): Le indica a JPA que la generación del valor del ID es delegada a la base de datos. IDENTITY es la estrategia correcta para las columnas de identidad de SQL Server.

@Table(name = "productos"): Opcional. Permite especificar un nombre para la tabla si no quieres que se base en el nombre de la clase.

@Column(name = "nombre_producto", length = 100): Opcional. Permite personalizar los detalles de la columna, como su nombre, longitud, si puede ser nula, etc.

Toda clase marcada como **@Entity** **debe tener un constructor vacío** (`public Clase() {}`). Hibernate usa un mecanismo llamado *Reflection* para crear los objetos desde la base de datos, y necesita ese constructor para instanciarlos antes de llenarlos con datos.

Mapeo de Herencia Reutilización sin Tablas (@MappedSuperclass)

Es una anotación de diseño arquitectónico. Resuelve el problema de la herencia en bases de datos cuando la clase Padre es netamente **técnica** y no un concepto del negocio.

- **No existe en SQL:** Hibernate NO creará una tabla para la superclase.
- **Herencia de Columnas:** Las tablas de las clases hijas tendrán sus propias columnas PLUS las columnas definidas en la superclase mapeada.

```
EL PADRE TÉCNICO (Sin tabla propia)
@MappedSuperclass
public abstract class RegistroAuditoria {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "fecha_creacion", updatable = false)
    private LocalDate fechaCreacion;

    // ... Constructores vacíos y Getters/Setters
}
```

Preguntas Frecuentes

¿Por qué usamos el objeto envolvente Long para el ID en lugar del tipo primitivo long?

Un tipo primitivo long en Java nunca puede ser nulo (su valor por defecto es 0). Un objeto Long sí puede ser nulo. Para Spring Boot, un objeto con ID null significa "Este objeto es nuevo, debo hacer un INSERT". Si el ID tiene un número, Spring asume que ya existe en la BD y hará un UPDATE. Si usas primitivos, Spring se confunde.

¿Puedo usar polimorfismo para buscar objetos de una @MappedSuperclass? (Ej. repositorio.buscarTodos())

NO. Como @MappedSuperclass no genera una tabla, no puedes hacer consultas directas a ella. No puedes decirle a JPA "Tráeme todas las Entidades Base". Solo puedes consultar a las clases hijas (Cliente, Producto), ya que ellas sí son entidades reales con tablas.

Dependencias genéricas esenciales

Al crear un proyecto Spring Boot, las siguientes dependencias son casi siempre las primeras que se incluyen. Las agregaremos en nuestro pom.xml.

- **spring-boot-starter-web**: El "motor" para construir aplicaciones web y APIs REST. Incluye el servidor embebido (Tomcat) y las librerías necesarias para crear controladores.
- **spring-boot-starter-data-jpa**: Facilita la interacción con bases de datos relacionales usando **JPA (Java Persistence API)**. Esto nos permite manejar objetos Java en lugar de escribir consultas SQL complejas. Detrás de escena, utiliza una implementación popular como Hibernate.
- **spring-boot-starter-dev-tools**: Una dependencia muy útil para el desarrollo. Permite el **reiniciado automático** de la aplicación cada vez que detecta un cambio en el código, lo que ahorra tiempo valioso.
- **Conector de base de datos**: Para conectar Spring Boot con una base de datos específica, necesitamos su **driver JDBC**. Ejemplos:
 - mysql-connector-java (para MySQL)
 - postgresql (para PostgreSQL)
 - h2 (para bases de datos en memoria, ideal para desarrollo y pruebas)
 - MS SQL Server Driver

Conexión a una Base de Datos H2

//Almacenar en un archivo .db

//spring.datasource.url=jdbc:h2:file:./data/ecommerce;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE

==== H2 DATABASE ====

spring.datasource.url=jdbc:h2:mem:clinicabd;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE

spring.datasource.driver-class-name=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=

==== JPA / HIBERNATE ====

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

==== H2 CONSOLE ====

spring.h2.console.enabled=true

spring.h2.console.path=/h2-console

http://localhost:8080/h2-console

usando la URL: jdbc:h2:file:./data/ecommerce

Ejemplos Demostrativos

Vamos a configurar un proyecto y a codificar el patrón de herencia.

Paso 1: La Superclase Mapeada

```
import jakarta.persistence.*;
import java.time.LocalDateTime;

@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // updatable=false evita que alguien modifique la fecha de creación en el futuro
    @Column(nullable = false, updatable = false)
    private LocalDateTime creadoEn;

    public BaseEntity() {
        this.creadoEn = LocalDateTime.now(); // Se asigna automáticamente al nacer el objeto
    }

    public Long getId() { return id; }
    public LocalDateTime getCreadoEn() { return creadoEn; }
}
```

Paso 2: La Entidad Concreta (Hija)

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name = "inventario_equipos")
```

```
public class EquipoTecnologico extends BaseEntity {
```

```
    @Column(nullable = false, unique = true, length = 50)
```

```
    private String numeroSerie;
```

```
    private String marca; // Si no ponemos @Column, JPA lo crea como VARCHAR(255) permitiendo nulos
```

```
    public EquipoTecnologico() {} // Constructor vacío obligatorio de JPA
```

```
    public EquipoTecnologico(String numeroSerie, String marca) {
```

```
        super(); // Llama al constructor del padre que asigna la fecha
```

```
        this.numeroSerie = numeroSerie;
```

```
        this.marca = marca;
```

```
    }
```

```
    // ... Getters y Setters
```

```
}
```

Práctica acompañada

Con el ejemplo anterior creado, continua con los siguientes pasos:

Configurar la conexión: Abre `src/main/resources/application.properties` y añade las propiedades de conexión a tu base de datos.

Crear la entidad Producto: Crea la clase `Producto.java` en un paquete `com.tuempresa.inventario.model`. Anótala con `@Entity` y `@Id @GeneratedValue(strategy = GenerationType.IDENTITY)`. Añade campos como nombre y precio.

```
@Entity
public class Producto {
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Id
    private Long id;
    private String nombre;
    private int precio;
}
```

Agregar constructores, getter y setter, y finalizamos con anotaciones para la base de datos. Además crea una clase `Categoria`.

Afianzamiento (Implementando @MappedSuperclass)

Ahora, apliquemos arquitectura.

1. Crea una clase abstracta EntidadSegura. NO le pongas @Entity.
2. Añádele @MappedSuperclass.
3. Córtale el id de tu clase Categoria (creada arriba) y pégalo en esta nueva clase base (con sus anotaciones @Id y @GeneratedValue).
4. Añade a EntidadSegura un atributo Boolean estadoActivo (para simular borrado lógico). Asigne el valor true por defecto.
5. Haz que Categoria haga extends de EntidadSegura.
6. Haz que Producto también extienda de EntidadSegura y tenga un atributo Double precio.

Dinámica de "Mob Debugging":

Código con un error arquitectónico sutil.

```
public class AuditoriaBase {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String usuarioCreador;  
}
```

```
@Entity  
public class Comentario extends AuditoriaBase {  
    private String texto;  
}
```

Relaciones entre Entidades

Así es como mapeamos las claves foráneas del mundo SQL al mundo de los objetos.

- **@ManyToOne**: La relación más común. Se pone en la entidad "hija". Por ejemplo, muchos Productos pertenecen a una Categoría. La clase Producto tendría un campo Categoría anotado con @ManyToOne. Esto crea la columna de clave foránea en la tabla de productos.
- **@OneToMany**: Es la otra cara de la moneda. Se pone en la entidad "padre". La Categoría tendría una List<Producto> anotada con @OneToMany. El atributo mappedBy es crucial aquí para decirle a Hibernate que la relación ya está gestionada por el lado @ManyToOne, evitando la creación de tablas o columnas extra.
- **Cascading (cascade = CascadeType.ALL)**: Una opción muy útil que le dice a Hibernate: "Si guardo una Categoría, guarda también todos los Productos en su lista. Si la borro, borra también sus productos". Simplifica las operaciones.

JPA nos permite modelar las relaciones de bases de datos directamente con anotaciones. La más común es la relación "muchos a uno".

Conceptos Clave de Relaciones:

- **Cardinalidad:** Describe la relación entre dos tablas:
 - **Muchos a Uno (@ManyToOne):** Muchos Productos pueden pertenecer a una Categoría.
 - **Uno a Muchos (@OneToMany):** Una Categoría puede tener muchos Productos.
 - **Uno a Uno (@OneToOne):** Un Usuario tiene un PerfilDeUsuario.
 - **Muchos a Muchos (@ManyToMany):** Muchos Estudiantes pueden estar en muchos Cursos.
- **Direccionalidad:**
 - **Unidireccional:** Solo una entidad "conoce" la relación. Por ejemplo, Producto conoce a Categoría, pero Categoría no sabe qué productos tiene.
 - **Bidireccional:** Ambas entidades se conocen mutuamente. Producto conoce a su Categoría, y Categoría tiene una lista de sus Productos.

Ejercicio Independiente Aplicado

Estás desarrollando un sistema de foros. Debes diseñar las tablas para los Temas y las Respuestas, asegurando las mejores prácticas de auditoría de base de datos.

Instrucciones de la Actividad:

1. En tu proyecto de Spring Boot, crea un paquete llamado models.
2. Crea la clase ModeloGenerico y decórala con @MappedSuperclass.
3. Debe contener: id (autoincremental), fechaModificacion (LocalDate) y version (Integer).
4. Crea la entidad UsuarioForo que extienda del genérico. Atributos: nickname (único, no nulo, máx 20 caracteres).
5. Crea la entidad Publicacion que extienda del genérico. Atributos: contenidoTexto (no nulo).
6. Asegura que todas tengan su constructor vacío y getters/setters.
7. Haz un commit con el mensaje feat: implementacion de MappedSuperclass en foros y sube tu rama a GitHub.

Criterios de Logro Esperados:

1. **Persistencia correcta:** Al arrancar el proyecto, la consola no arroja excepciones de Hibernate.
2. **Mapeo exacto:** La base de datos debe contener solo 2 tablas (UsuarioForo y Publicacion). No debe existir una tabla ModeloGenerico.
3. **Restricciones:** El campo nickname fue configurado correctamente usando @Column(unique = true, nullable = false, length = 20).

CESDE | comfama